CNN-GRU Fusion Model for Synchronous Motor Electrical Fault Classification

Julian P. Guda

1.0 Introduction

Synchronous motors are used in almost every facet of industry from manufacturing systems, wind turbines, electric transportation, and propulsion systems, therefore reliable fault diagnosis is important whenever downtime, equipment damage or unsafe operation carries significant costs. Sun et al. motivates this clearly: electrical faults in synchronous motors can reduce efficiency, degrade reliability, and damage surrounding equipment,[1][2] which makes automated diagnosis and monitoring a relevant engineering problem[1].

A major challenge in this area is the lack of a common benchmark for fair comparison. To address this, Sun et al. introduced a public dataset for synchronous motor electrical fault diagnosis together with CNN and LSTM reference classifiers. The dataset includes five electrical fault types and normal operation recorded from a real inverter-driven synchronous motor and was explicitly proposed as a benchmark for future classifier development[1]. The link to the dataset is provided here:

This work develops a CNN-GRU fusion model for the same six-class classification task. More specifically, the task is window-level classification of multichannel time-series segments into one of six labels: VREC, OP, REVD, 2PSC, 1PSC, or NF. The model combines three one-dimensional convolutional blocks for local feature extraction with a gated recurrent unit (GRU) module for temporal modelling. The classifier is evaluated across five window sizes: 100, 200, 400, 600, and 800 time points. Under the present preprocessing evaluation pipeline, the model achieves 100% window-level accuracy and 100% macro F1-score for all tested window sizes.

However, a channel ablation study shows that, under the same pipeline, even restricted channel subset can also achieve perfect classification. Therefore, the more defensible conclusion is that the benchmark becomes highly separable under the present preprocessing and evaluation choices. The report focuses on both the classifier study as well as a critique of the benchmark's sensitivity. Rather than claiming that the CNN-GRU architecture alone outperforms the published baselines, the stronger conclusion is that the benchmark is highly sensitive to preprocessing and evaluation choices. Accordingly, this report contributes both a high-performing classifier and a critical assessment of benchmark difficulty under the implemented pipeline.

2.0 Related Work

Motor fault diagnosis methods can broadly be divided into traditional signal-processing approaches and data-driven learning approaches. The benchmark paper reviews traditional methods such as Fourier transforms, matched filters, and wavelet transforms.[3][4] These methods can be effective in specific settings, but they have notable drawbacks. Fourier and wavelet methods may be computationally expensive for real-time deployment, and the Fourier-based or matched-filter approaches are typically better suited to steady-state conditions than dynamic or transient faults.[2] Wavelet methods also require predefined frequency-band selection that may vary across applications.[3]

Classical machine learning has also been applied to synchronous motor fault diagnosis. The benchmark paper discusses support vector machines and conventional neural-network-based methods, while also noting their limitations. SVM[5] performance depends strongly on parameter selection and optimization while shallow neural networks may require large amounts of data and may be prone to overfitting

Recent work has increasingly used deep learning because of its ability to learn discriminative features directly from raw or lightly processed sensor data. The benchmark paper cites CNNs[6], RNNs[7], Attention-Based models and feature-fusion approaches for related machine-fault tasks. It also introduces two reference classifiers of its own: a CNN and an LSTM. At 800 recording points, the paper reports CNN accuracies of 97.05% for stator voltage, 98.38% for stator current, 98.97% for rotor current, and 43.81% for rotor speed.

The paper further shows that certain two-feature combinations can reach 100% accuracy for the CNN baseline, especially stator voltage with rotor current and stator current with rotor current. This suggests that feature fusion is already highly effective on the benchmark and motivates the use of a hybrid model that combines convolutional feature extraction with recurrent temporal modelling.

The present work builds on that direction by using a CNN-GRU architecture. Compared with an LSTM, a GRU simplifies the LSTM architecture by merging the forget and input gate into an update gate. It also does not maintain a separate cell state; therefore, it uses fewer gating operations and fewer recurrent parameters than an LSTM. In this study,

the recurrent stage is operating on a shorter CNN-produced feature sequence. Thus, the GRU was chosen as it preserves temporal modelling while reducing recurrent complexity.

3.0 Methods

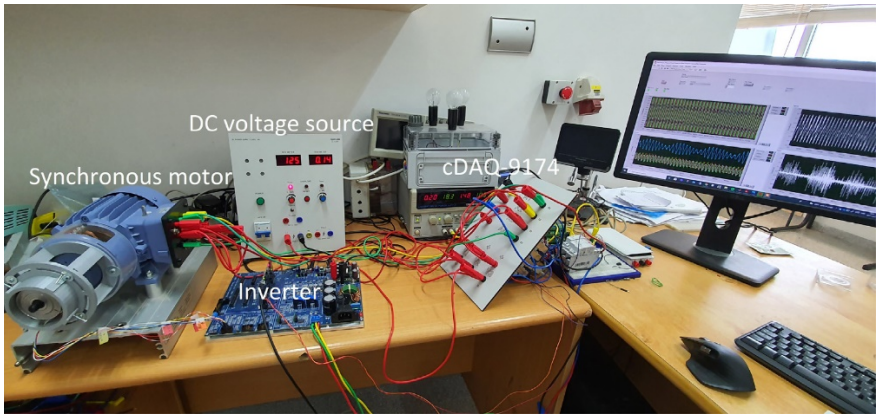

Figure 1 – Experiment Setup

3.1 Dataset and Experimental Setup

The dataset used in this work is generated from the synchronous motor benchmark setup of Sun et al. Their experimental setup consists of a 125V DC source feeding a three-phase two-level inverter implemented using the Texas Instruments TMDXIDDK379D development kit, which drives a 300W synchronous motor. The measured signals are recorded using a National Instrument cDAQ-9174 system. The paper reports a motor nominal speed of 1345 rpm, nominal frequency of 50 Hz, stator nominal voltage of 230V delta/ 400V wye, and nominal current of 0.75A.

The benchmark includes five electrical faults: open phase between inverter and motor, short circuit/leakage current between two phases, short circuit/leakage current in phase-to-neutral, rotor excitation voltage disconnection, and variation of rotor excitation current. For data acquisition, each experiment was repeated many times and the fault onset occurred at a random time point within the recording.[1]

In the present implementation, six pre-processed dataset files were loaded and concatenated into a single dataset. The dataset-loading code reads from train and label data from the pre-processed files from corresponding to NF, OP, 2PSC, 1PSC, REVD, and VREC. The combined dataset contains 3,392 Experiments. Each experiment contains 10,000 time points and 9 channels. The class distribution is imbalanced: VREC has 322 experiments, OP has 703, REVD has 518, 2PSC has 753, 1PSC has 767, and NF has 329, corresponding to an imbalance ratio of approximately 2.38 between the largest and smallest classes.

3.2 Channel Interpretation

Based on inspection of the benchmark data structure and EDA plots, the channel interpretation used in this project is summarized as follows:

- Channels 0-2: stator voltage A/B/C
- Channels 3-5: stator current A/B/C
- Channel 6: rotor current
- Channel 7: rotor speed
- Channel 8: undocumented periodic channel, treated as an alternate rotor-speed signal

The interpretation of channel 8 is inferred rather than explicitly documented in the paper, so results involving all 9 channels should be read with the uncertainty in mind. The EDA-based signal interpretation is discussed in the Results section.

3.3 Preprocessing Pipeline

The code uses train-test split with stratification and random seed to create an 80/20 split, producing 2,713 training experiments and 679 held-out experiments. This order is done so to prevent windows from the same original experiment from appearing in both partitions, which would otherwise introduce leakage and inflate the reported performance.

After splitting, channel-wise z-score normalization is computed from training partition only; then applied to both the training and held-out data. The channel means and standard deviations are computed by reshaping the training set across all experiments and time points, then applying the resulting statistics to both splits. This step prevents direct

leakage of held-out distribution statistics into the training preprocessing.

The normalized experiments are then converted into fixed-length windows through a dataset wrapper which prepares samples on demand for training. In this main pipeline, the dataset defaults to the stride being half of the window size, meaning 50% overlap unless a different stride is passed explicitly. The dataset does not pre-store the window tensors. Instead, it builds an index map of experiment and start time pairs and slices each window on demand; this was done to minimize memory usage due to actual memory constraints. Each window is transposed to the channels and timestamps before being returned to the model.

To reduce the effect of class imbalance during training, the data loader uses weighted sampling (WeightedRandomSampler) from PyTorch library which ensures that minority classes are seen more frequently while majority classes are relatively less frequent. Weighted cross-entropy loss is also used, where class weights are computed inversely from class frequencies in the training labels; this heavily penalizes misclassifying minority-class samples. Both weighted sampling and cross-entropy loss ensures that the bias towards the majority classes are minimized at the sampling and optimization stage.

Compared with the benchmark paper's reference pipeline our preprocessing is done on the experiment-level whereas the reference is disturbance centred. In the paper's dataset code, each sample is cropped from a fixed region around index 400 using, and only the selected feature subset is retained. As a result, the published models are trained on pre-aligned fault-centred segments rather than windows extracted across the full recording. By contrast, our pipeline loads the pre-processed signal data, performs a stratified experiment-level split before windowing, applies training-only normalization, and then constructs windows from the full recordings. Our main dataset class also uses overlapping sliding windows by default, which generates many windows per experiment instead of a single curated disturbance crop. This difference in preprocessing is likely the reason why the classification task appears easier under our setup than the reference paper's pipeline.

3.4 CNN-GRU Architecture

The proposed model is implemented as a hybrid 1D CNN-GRU network. The model code defines a reusable CNN block composed of Conv1d, BatchNorm1d, ReLU, MaxPool1d, and dropout. Three such blocks are stacked, mapping the input channels 9 to 32, 32 to 64 and 64 to 128 with a kernel size of 7 and same padding behaviour.

After three max-pooling stages, the temporal dimension is reduced by a factor of eight. The convolutional output is then permuted from (batch, features, time) to (batch, time, features) and passed through a two-layer GRU with hidden size 256 and dropout 0.3. The output from the final time step is fed into a classifier head consisting of Linear (256 to 128), ReLU, dropout 0.4 and Linear (128 to 6). The resulting model has 799,238 trainable parameters across the tested window sizes.

This architecture was chosen to combine two complementary inductive biases. The convolutional front-end is suited to extracting local waveform patterns and cross-channel relationships, while the GRU captures temporal dependencies in the feature sequence produced by the CNN. Table 1 outlines the architecture:

Component	Details
CNN Block 1	Conv1d (9 to 32, kernel=7, same-pad), BatchNorm1d, ReLU, MaxPool1d(2), Dropout(0.3)
CNN Block 2	Conv1d (32 to 64, kernel=7, same-pad), BatchNorm1d, ReLU, MaxPool1d(2), Dropout(0.3)
CNN Block 3	Conv1d (64 to 128, kernel=7, same-pad), BatchNorm1d, ReLU, MaxPool1d(2), Dropout(0.3)
Temporal Reduction	Factor of 8 times after three max-pooling stages
GRU	2-layer GRU, hidden size=256, dropout=0.3; final time-step output used
Classifier Head	Linear(256 to 128), ReLU, Dropout(0.4), Linear(128 to 6)
Total Parameters	Approx. 799,238 (consistent across all window sizes)

Table 1: CNN-GRU Architecture

3.5 Training Procedure

The training script is implemented in PyTorch and uses CUDA if available, otherwise CPU. Reproducibility is addressed by fixing both PyTorch and NumPy random seeds to 42. The default configuration uses batch size 64, 10 training epochs, learning rate 10^{-3}, weight decay 10^{-4}, 9 input channels, and 6 output classes; since the model converged rather fast, the number of epochs was reduced from 30 to 10.

Training uses Adam optimization, weighted cross-entropy loss and a ReduceLROnPlateau scheduler with factor 0.5 and patience 5. During training, the script evaluates the held-out split at the end of each epoch, steps the scheduler on held-out loss, and saves the checkpoint with the best held-out accuracy. Strictly speaking, this means the held-out split is functioning as validation-like split rather than a completely untouched final test set. That distinction is important when interpreting the reported performance.

4.0 Results

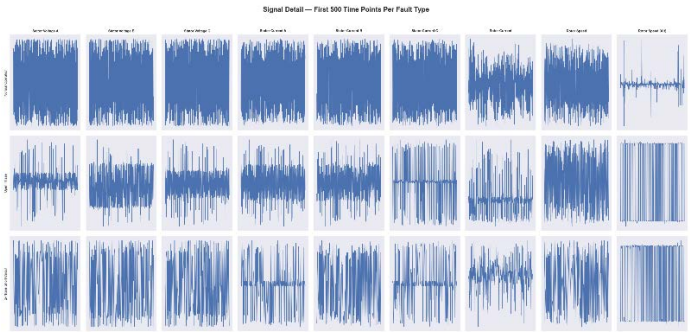

Figure 2 – First 500 time points of Signal (truncated)

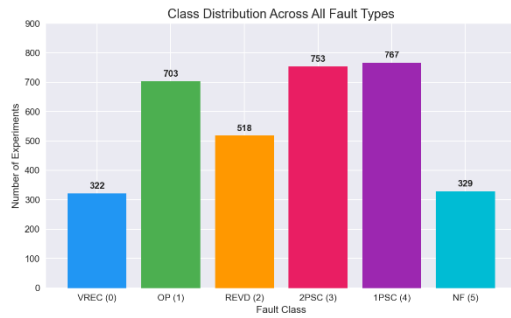

Figure 3 – Class Distribution of Fault Types

4.1 Exploratory Data Analysis

The class-distribution figure confirms moderate imbalance but not an extreme skew. The largest classes, 1PSC and 2PSC are roughly 2.3 to 2.4 times larger than the smallest class, VREC. The EDA signal-comparison plots show that the fault classes are visually distinct across several channels. In particular, voltage and current channels exhibit strong waveform distortions or amplitude changes in the phase-related faults, while the rotor-current-related faults show abrupt changes in the rotor current and sometimes associated changes in other channels.

This observation is consistent with the examples shown in the benchmark paper, which illustrate clear differences between normal operation and the studied fault cases. For example, the paper shows visible distortions in stator voltages, stator currents, and rotor current for open-phase and short-circuits faults, and strong reductions in rotor current for rotor excitation faults. The visual separability seen in the EDA helps explain why the classification task becomes very easy under the present preprocessing pipeline.

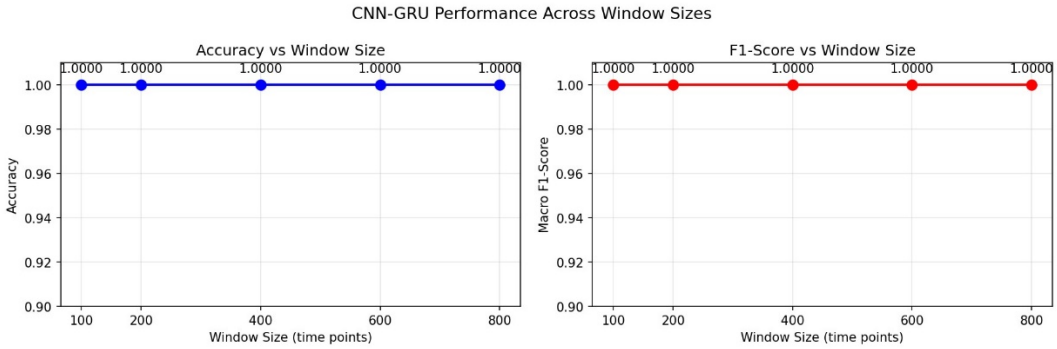

Figure 4 – CNN-GRU Performance Across Window Sizes (100, 200, 400, 600, 800)

4.2 Window-Size Study

The CNN-GRU was evaluated at window sizes of 100, 200, 400, 600 and 800 points. Across all five settings, the reported window-level accuracy and macro F1-score were 1.0000. The confusion matrices for each window size are perfectly diagonal, with zero observed misclassifications in both raw-count and normalized views. The training history plots show that held-out accuracy reaches 1.0 almost immediately and remains there for the rest of training

The results suggests that the class signature are persistent and strong enough to be recognized even in relatively short windows. Under the current pipeline, increasing the temporal context from 100 to 800 points does not improve classification performance because the task was already saturated at the shortest tested window length.

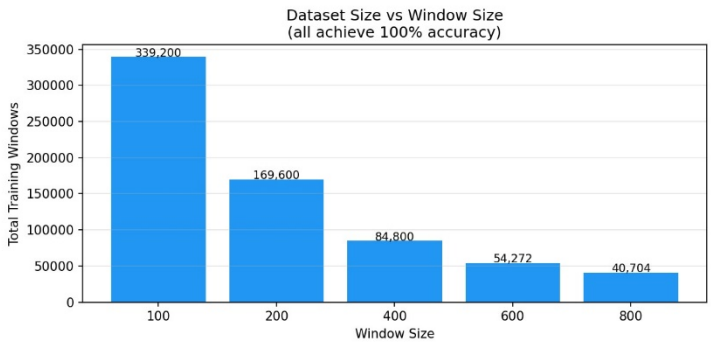

Figure 5 – Dataset Size vs Window Size

4.3 Window Size Versus Dataset Cardinality

Although performance is identical across all tested windows, the window size still affects the number of generated training samples. The total number of training windows decreases from 339,200 at 100 points to 169.600 at 200 points, 84,800 at 400 points, 54,272 at 600 points, and 40,704 at 800 points. This is the expected consequence of longer windows producing fewer segments from the same underlying recordings.

The practical implication is that window size should not be discussed only in terms of accuracy. A smaller window may be preferable when the lower latency is desired, since it enables earlier decisions. A larger window may be preferable when reducing dataset cardinality and simplifying training logistics is more important.

4.4 Classification Performance

The main evaluation script computes overall accuracy, per-class precision, recall, F1-score, and confusion matrices at the window level. The saved results for the representative 800-point case show 100% overall accuracy and perfect per-class metrics, with all six classes achieving precision, recall and F1-score of 1.000. The confusion matrix is perfectly diagonal for the held-out windows.

At face value, this indicates complete separability of the benchmark under the present pipeline. However, the result should be interpreted carefully because the held-out split was monitored during training and because the window-level evaluation counts many correlated windows derived from the same experiments.

Feature Subset	Channels Used	CNN-GRU Best Acc.	Sun et al. CNN @ 800 pts
Stator voltage only	0, 1, 2	1.0000	0.9705
Stator current only	3, 4, 5	1.0000	0.9838
Rotor current only	6	1.0000	0.9897
Rotor speed only	7	1.0000	0.4381
Stator voltage + Rotor current	0–2, 6	1.0000	1.0000
Stator current + Rotor current	3–5, 6	1.0000	1.0000
All 9 channels	0–8	1.0000	N/A (Not Reported)

Table 2 – Ablation Study Results

4.5 Channel Ablation Study

The channel ablation code evaluates multiple feature subsets aligned with the combinations emphasized in the benchmark paper: stator voltage only, stator current only, rotor current only, rotor current only, stator voltage plus rotor current, stator current plus rotor current and all 9 channels. The ablation uses the same experiment-level split and training-only normalization as the main pipeline, but explicitly uses window size as 800 and stride size as 800 so there are no overlapping windows.

The results show that every tested channel subset reached a best accuracy of 1.0000. The benchmark paper by contrast reports CNN accuracies at 800 points of 97.05% for stator voltage, 98.38% for stator current, 98.97% for rotor current, and 43.81% for rotor performance, whereas the present ablation reaches perfect classification.

This finding is central to the interpretation of the project. Since even single-feature subsets achieve perfect accuracy under the implemented pipeline, the observed gains over the benchmark baselines cannot reasonably attributed to the CNN-GRU architecture alone. Instead, they are much more likely to result from differences in preprocessing, segmentation, overlap strategy, evaluation protocol, or some combination of those factors.

5.0 Discussion

The reported results demonstrate that the CNN-GRU can solve the present version of the benchmark classification problem perfectly at the window level. The EDA supports this outcome: many of the classes appear visually separable and the benchmark paper itself shows strong waveform differences among the faults. In that sense, the CNN-GRU is clearly capable of learning the class structure in the available data.

At the same time, the ablation study reveals that the benchmark under this pipeline is easier than initially expected. If stator voltage only, stator current only, rotor current only, rotor speed only, mixed subsets and all channels all achieve perfect best accuracy, then the hybrid CNN-GRU architecture is not the sole explanation for the final result. The more convincing conclusion is that the task as presently framed is highly separable.

There are several possible reasons. First, the preprocessing differs from the benchmark paper's original reference setup. Second, the main training pipeline uses overlapping windows, which increases the number of correlated samples. Third, the held-out split is used during training for monitoring and checkpoint selection, which makes it more validation-like than a final blind test set. Fourth, the signals may represent sustained fault states rather than subtle incipient faults or onset-only detection. Any of these factors can reduce task difficulty.

Therefore, this study cannot be interpreted as a direct comparison to the CNN and LSTM baselines. Instead, the CNN-GRU achieves its performance under the implemented data pipeline which is different from the benchmark. The accompanying ablation study reveals that the benchmark difficulty is sensitive to how the data is prepared.

6.0 Conclusion

This work presented a CNN-GRU fusion model for synchronous motor electrical fault classification using the public synchronous motor benchmark introduced by Sun et al. The model combined three one-dimensional convolutional blocks with a two-layer GRU and a fully connected classifier head. Under the implemented preprocessing and evaluation pipeline, the model achieved 100% window-level accuracy and 100% macro F1-score across all tested window sizes from 100 to 800 points.

The most important finding, however, is not simply the perfect accuracy. The channel ablation study showed that, under the same pipeline, even highly restricted feature subsets can also achieve perfect best accuracy. This indicates that the benchmark becomes highly separable under with the chosen method to prepare the data before being fed into model, and that the apparent gains over the benchmark paper's published baselines cannot be attributed to the CNN-GRU architecture alone.

The study therefore has two main takeaways. First, the CNN-GRU is a strong and effective classifier for this benchmark under the present pipeline. Second, benchmark comparisons in this problem are highly sensitive to data-splitting, windowing, and evaluation choices. Future work should include a strict train-validation-test protocol, closer reproduction of the benchmark paper's original preprocessing, experiment-level aggregation, ablation of individual channels with a documented channel map, and testing on more realistic scenarios such as gradual fault development or online onset detection.

References

[1] Z. Sun, R. Machlev, Q. Wang, J. Belikov, Y. Levron, and D. Baimel, "A public data-set for synchronous motor electrical faults diagnosis with CNN and LSTM reference classifiers," Energy AI, vol. 14, p. 100274, 2023. **Dataset**

[2] Choi S, Haque MS, Tarek MTB, Mulpuri V, Duan Y, Das S, Garg V, Ionel DM, Masrur MA, Mirafzal B, Toliyat HA. Fault diagnosis techniques for permanent magnet AC machine and drives—A review of current state of the art. IEEE Trans Transp Electrif 2018;4(2):444–63.

[3] Ebrahimi BM, Faiz J, Roshtkhari MJ. Static-, dynamic-, and mixed eccentricity fault diagnoses in permanent-magnet synchronous motors. IEEE Trans Ind Electron 2009;56(11):4727–39.

[4] Ebrahimi BM, Faiz J. Feature extraction for short-circuit fault detection in permanent-magnet synchronous motors using stator-current monitoring. IEEE Trans Power Electron 2010;25(10):2673–82.

[5] Pietrzak P, Wolkiewicz M. Application of support vector machine to stator winding fault detection and classification of permanent magnet synchronous motor. In: International power electronics and motion control conference. 2021, p. 880–7.

[6] Wang C-S, Kao I-H, Perng J-W. Fault diagnosis and fault frequency determination of permanent magnet synchronous motor based on deep learning. Sensors 2021;21(11):3608.

[7] Lee H, Jeong H, Koo G, Ban J, Kim SW. Attention recurrent neural network-based severity estimation method for interturn short-circuit fault in permanent magnet synchronous machines. IEEE Trans Ind Electron 2020;68(4):3445–53.